

Project Report: Identification of canonical neural events during continuous gameplay of an 8-bit style video game

Sana Hafeez, Hadi Ismail, Okan Mazlum

SS 2026

Introduction

This report documents an attempt to reproduce the main findings of *Identification of canonical neural events during continuous gameplay of an 8-bit style video game* by Cavanagh and Castellanos (2016), for the module *Signal processing and Analysis of human brain potentials (EEG)*. [1] We use the study's publicly shared EEG dataset, OpenNeuro ds003517. [2] The accompanying project repository provides the code and reproduction instructions. [3]

The main question of this project is to see whether we can reproduce the results of the paper using a different preprocessing and analysis pipeline, which would strengthen the robustness of the findings in the original paper.

Original paper

The motivation of the original paper is the issue of generalizability in classical EEG experiments. In order to minimize artifacts, EEG tasks are often designed with isolated stimuli in carefully controlled environments. These however can be difficult to interpret in a complex, continuous environment. The authors developed the space-shooter game *Escape from Asteroid Axon* to study these complex events while retaining precise event timing. Before playing, subjects completed two established exemplar tasks: oddball target detection and gambling wins/losses. The authors compared the EEG responses statistically and trained subject-specific classifiers on these exemplar tasks, then transferred their learned patterns to gameplay. [1]

The main finding was that wall crashes, enemy crashes and missile hits showed a transfer bias toward oddball-rare patterns during the P3 period. Missile hits also resembled gambling wins, whereas crashes did not reliably resemble gambling losses. The authors interpreted these results as evidence that some signatures from the exemplary tasks associated with salience and reward generalize to gameplay; the expected punishment-related result (gambling loss transfers to crashes) could not be established. [1]

Dataset description

The dataset contains 17 subjects (11 men; ages 18–39, mean 20.94 years), reported as right handed. Each subject has two runs under the BIDS task name ContinuousVideoGamePlay. Run 01 contains oddball and gambling tasks; run 02 contains gameplay. Full recording durations average 777.78 s and 1,976.49 s respectively and include periods outside the active tasks. EEG was sampled at 500 Hz with CPz as acquisition reference. There are 63 recorded EEG channels plus bipolar VEOG and HEOG; CPz is not independently recorded. All of the supplied electrode positions match the BESA standard-10-5-cap385.elp montage [2]

Oddball

Participants mentally counted rare enemy-spacecraft images without a button response. The intended design contained 160 standard and 40 rare images. Each image was shown for 500 ms, followed by a random 1–2 s interval. Rare events occurred after at least three but before ten standards. [1] The duration of this task averaged 6.68 min across the 17 subjects.

Gambling

Participants chose one of two doors using the left/right gamepad controls. Each trial resulted in a win of 75 credits or a loss of 60 credits. The intended design contained 40 wins and 40 losses. The doors remained on screen for up to 4 s or until a choice. Feedback was then presented immediately for 500 ms, followed by a 1 s interval. The paper gives a mean task duration of 2.86 min. [1]

Escape from Asteroid Axon

Participants steered a ship and fired missiles, avoided wall/enemy crashes, collected stars to restore health and collected ammunition. An empty health bar ended the round; another round could be started by a button press. Game events include shots, star collections, ammunition collection, wall crashes, enemy crashes and missile hits. Gameplay was intended to last 30 min. [1]

Dataset exceptions and recording boundaries

- All 17 subjects have 200 oddball events, but some early recordings contain 37–44 rare events rather than exactly 40.
- Subjects 002, 003 and 005 have no gambling-outcome events, which is also reported in the original paper.
- Subject 001 has only 18 win and 17 loss events; subject 006 has 120 wins and 120 losses. The remaining subjects with gambling events have the intended 40 wins and 40 losses.
- Subject 001 has the extended gameplay session: approximately 45.9 min through the last logged game event, increasing the mean duration across subjects to 31 min.
- While all other subjects have a median of 90s of inactivity before actual gameplay starts, for subjects 003 and 008, the first 41 s and 35 s of gameplay in their run 2 are missing respectively. For subject 008, 11 s of their axon gameplay is instead located at the end of their oddball and gambling run

Preprocessing

We use MNE-BIDS-Pipeline because it is the workflow used in the course. The original authors used the MATLAB toolbox EEGLAB. Differences between these software ecosystems affect many implementation details and defaults (e.g. specific ICA implementation or channel interpolation algorithm). This chapter focuses instead on the main consciously made methodological choices and their reasons.

Initial loading attempt

Channel metadata

With a minimal configuration file, an attempt to load the original dataset fails during the pipeline's initial data-quality report. The error is `picks (None, treated as "data") yielded no channels`, consider passing `picks` explicitly. The reason is that all entries in the original `channels.tsv` type and units columns are `n/a`, making MNE believe there are no data channels for its calculations.

This motivated a preparation script `00_prepare_dataset/prepare_bids.py`, which creates a new “prepared” dataset with corrected metadata. The 63 scalp channels receive type `EEG`, `VEOG` and `HEOG` receive type `E0G`, and all channels receive units `uV`. The corresponding `*_eeg.json` channel type counts are adjusted accordingly.

Electrode coordinates

With channel types corrected, another warning appears on reexecution of the pipeline: `Other is not an MNE-Python coordinate frame for EEG data and so will be set to 'unknown'`. The original `*_coordsystem.json` declares `EEGCoordinateSystem = "Other"`, which is technically allowed in BIDS, but not correctly defined here. [4]

The supplied MATLAB scripts, `Convert2BIDS_ExAAx.m` and `ExAAx_Preprocess.m`, identify the used montage as `standard_BESA/standard-10-5-cap385.elc`. This allows us to extend the preparation script to additionally correct `*_electrodes.tsv` and `*_coordsystem.json`, transforming them into the corrected MNE head coordinates.

First inspection and visual delay

The prepared dataset allows an initial pipeline execution and inspection of minimally filtered EEG. However, this reveals yet another issue:

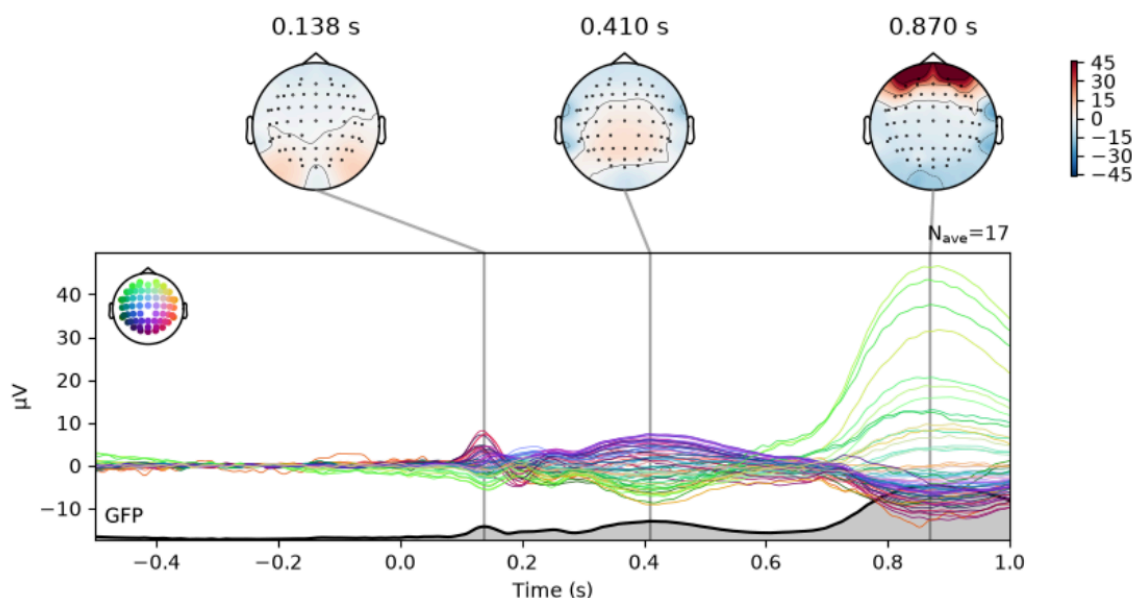


Figure 1: Initial grand average butterfly plot for rare oddball events with minimal filtering

The three observed peaks are the visual P1 (at 0.138s), P3/P300 at 0.410 s and eye artifacts at 0.870 s. However, all of the peaks are slightly too late - the P1 peak would usually be expected around 100ms and P3 at around 300-400ms. The cause of this is the 40ms visual delay also reported by the paper. Since the mne-bids-pipeline does not seem to have a configuration setting to shift event onsets, we also used the preparation script to shift all analysis-relevant event onsets 40 ms forward, to be in sync with the EEG recording.

Pipeline configuration

The table summarizes the main methodological pipeline differences.

Aspect	Original paper	This project
Bad-channel selection	Four ventral electrodes excluded, then FASTER	PyPREP including RANSAC; all threshold-based flags accepted after inspection; no default exclusions
Frequency filtering	0.01–100 Hz acquisition; 0.1–20 Hz ERP/classification	0.1–100 Hz continuous EEG with a 60 Hz notch; ICA fit at 1–100 Hz; ERP/classification at 0.1–40 Hz
Rereferencing	Initial average reference and CPz reconstruction; subsequent bad-channel interpolation	Average reference over the common non-bad EEG set; no CPz reconstruction

Aspect	Original paper	This project
Combining runs	Supplied MATLAB scripts suggest separate exemplar/gameplay decompositions	One ICA per subject across both runs
ICA training	-2...+2 s epochs around events	Evenly spaced 4 s windows without baseline or channel interpolation before fitting
ICA component selection	VEOG/HEOG correlations with manual verification	ICLabel proposals plus visual review
Bad-epoch handling	FASTER, channel interpolation and epoch rejection	AutoReject after ICA repairs channels and rejects analysis trials

Bad-channel selection

We chose PyPREP because it works natively with MNE data and provides several different channel-quality checks. We use NoisyChannels with default parameters to perform all included checks, including NaN/flat, deviation, high-frequency noise, correlation/dropout, SNR, PSD and RANSAC. [5] In exploratory runs, the flagged channels varied substantially. We therefore fixed the random seed at **2026** and increased RANSAC's `n_samples` to **1,000 random channel subsets** to improve reproducibility and sampling stability.

To check the flagged channels, we built a small inspection tool (`./review-channels` in `01_bad_channels/`) using MNE's data browser. It shows the flagged channel alongside neighbouring electrodes using raw or frequency-filtered EEG. For Correlation/dropout and RANSAC, PyPREP provides time intervals that triggered the algorithm to mark a channel as bad, these intervals can also be specifically examined in the tool.

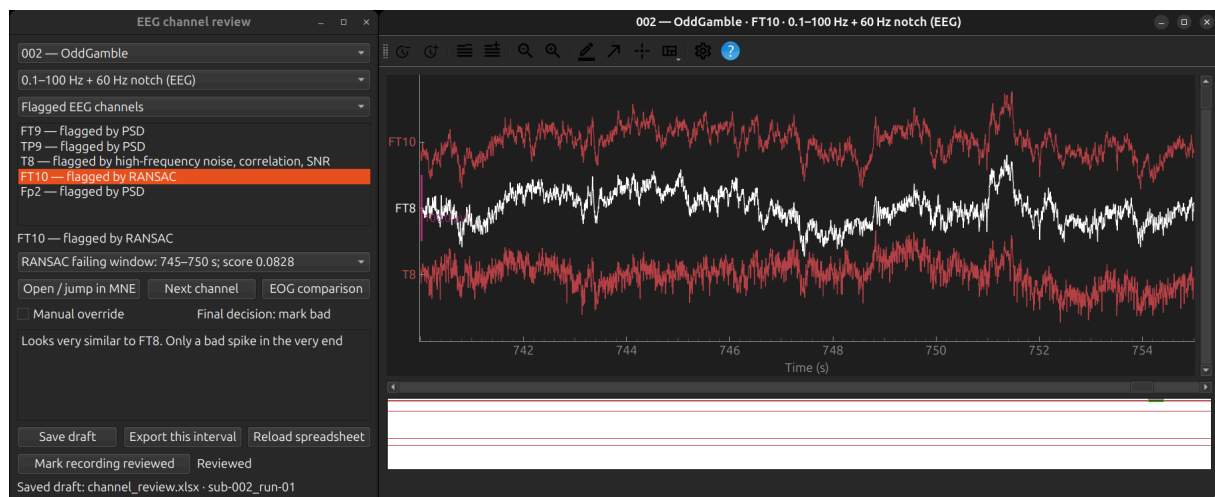


Figure 2: Analysis of run 1 of subject 002 in our tool, highlighting two channels that have been marked as bad

Our visual inspection led to three observations:

- For many flagged channels, the decision was easy to understand: the signal was visibly noisier than that of nearby electrodes (see T8 in Figure 2)
- Some flags were difficult to verify visually, especially RANSAC flags. Its prediction from multiple other electrodes is not necessarily apparent in an isolated trace comparison. (see FT10 in Figure 2, the picture shows the interval that decided on the channel exclusion)

- Some unflagged channels appeared problematic during inspection, but their detector scores lie just below the rejection threshold.

Given the project's scope and our limited experience with manual channel rejection, we adopted all PyPREP threshold-based decisions without manual overrides. Across all recordings, 68 channels were flagged. The median is two flags per recording (range 0–6); six recordings have none. CP2 is flagged most often, in seven recordings. The original paper also reports a median of two interpolated electrodes. [1]

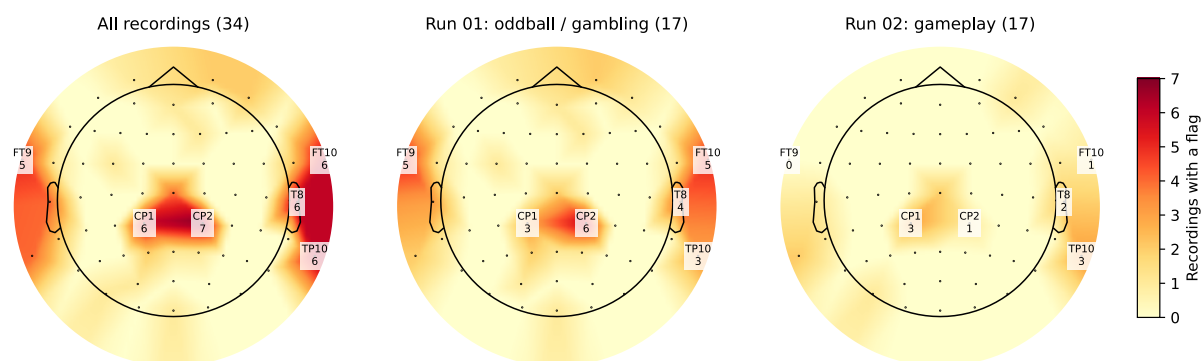


Figure 3: Frequency of PyPREP flags over all 34 recordings and separately by run. All panels use the same count scale. Labels identify electrodes flagged in at least five recordings overall.

The paper excluded FT9, TP9, TP10 and FT10 by default (i.e. 100% exclusion). PyPREP flags 21/136 of these ventral channels (15.4%), which is much lower than the default exclusion, but still high compared with 47/2,006 (2.3%) at the other 59 electrodes.

PyPREP runs in a separate script, `01_bad_channels/run_pyprep.py`, after the initial preparation and preliminary frequency-filtering and before the main ICA pipeline execution. The script writes the accepted flags directly into the `*_channels.tsv` sidecars of the prepared dataset. For later steps, it will be required that both runs of each subject share the same set of bad channels, which is why we will take the union of both lists here.

Frequency filtering

Continuous EEG is filtered to 0.1–100 Hz with a 60 Hz notch at the original 500 Hz sampling rate. For ERP/classification, we use a continuously filtered 0.1–40 Hz copy. The 40 Hz low-pass follows the filtering lecture's general recommendation to retain at least 40 Hz.

ICA training uses a separate 1–100 Hz copy to follow ICLabel's mandated settings, together with average reference and extended Infomax algorithm [6]. The original paper specifies 0.01–100 Hz acquisition and 0.1–20 Hz ERP/classification filtering, but no distinct ICA fitting filter. [1]

Rereferencing

The paper describes an initial average reference and CPz reconstruction before ventral-channel removal and FASTER cleaning, which is also supported by the MATLAB scripts [1]. To avoid known bad channels contributing to the average, this project instead references over the common accepted EEG set after bad-channel selection, (excluding EOG) and globally bad EEG channels. We do not reconstruct CPz.

Combining runs within subjects

We retain the pipeline's default behavior of fitting one ICA per task across all included runs. The original paper instead processed exemplar and gameplay recordings separately, as suggested by the MATLAB scripts.

Our motivation behind this was

1. The experiments were seemingly recorded within the same session, as evidenced by the start of Subject 008's gameplay being included in its first run (a nice side effect is that due to the internal concatenation of the recordings, this is no longer an issue)
2. Making the two different recordings more comparable for transfer analysis, by having them undergo the same ICA component removal
3. Providing more training epochs for ICA training, which could be especially useful when one task loses many windows in the autoreject

We consequently produce one saved ICA model per subject. As previously mentioned, this choice also requires the union of each subject's run-specific bad channels, as the installed pipeline fails if the runs' bad-channel lists differ.

Each run is filtered and windowed separately before the fitting epochs are concatenated, so windows never cross recording boundaries.

ICA training

An initial pilot used $-2...+2$ s epochs around selected events, similar to the paper's epoching description. However due to the large number of gameplay events, this produced extensive overlap of epochs, repeatedly presenting the same samples to ICA and increasing processing time. Local AutoReject also removed large amounts of exemplar data. We experimented with various configuration parameters, but eventually settled on using `task_is_rest = True` to create evenly spaced, non-overlapping four-second fitting windows. We also downsampled to 250 Hz to save processing time. The final approach was much faster and retained a very similar decomposition to the first attempt (`ica_pilot` vs `ica_final_full`).

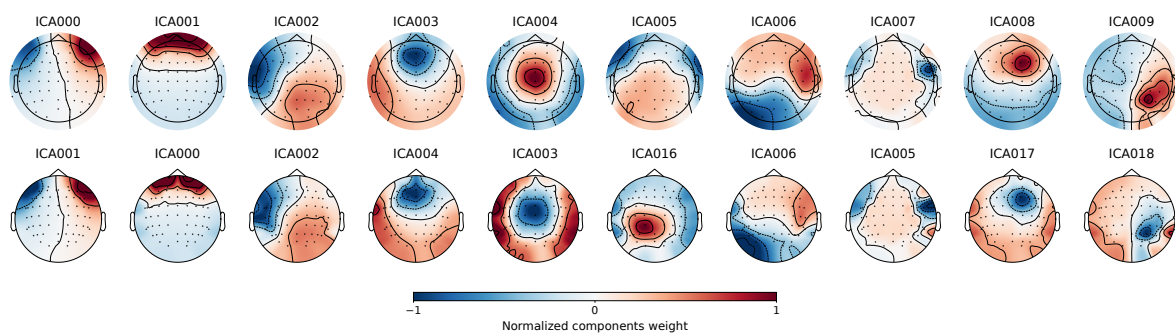


Figure 4: ICA decomposition for subject 7 in `ica_pilot` (top row) and `ica_final_full` (second row).

Components have been reordered for the second row to show their closest match

We used the regular extended Infomax for all our preprocessing, despite the Picard extended Infomax algorithm theoretically featuring faster convergence. This was chosen due to a controlled subject-001 comparison, where regular Infomax took only 8 min 49 s, while picard took 16 min 47 s.

Across all 17 subjects, 9,714/11,655 (83.35%) fitting windows were retained (85.7% oddball, 87.2% gambling and 87.3% gameplay), providing 24.3–54.5 min of training EEG per subject. The models contain 56–62 components per subject (according to available channels).

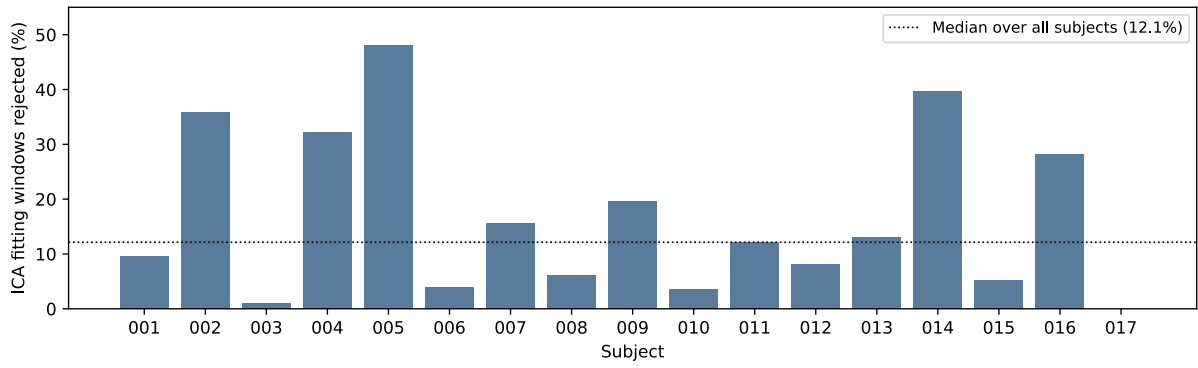


Figure 5: AutoReject rejection of ICA fitting windows by subject

Rejection of training epochs varied strongly across subjects. Subject 014 retained only 4/33 windows attributed to gambling, but still had more than 24 min of training data overall. Due to the high rejection rates in subject 005 and 014, we experimented with threshold-based rejection at $500\mu\text{V}$, which retained 94.7% and 96.7% of their fitting windows. However, this did not yield a better ICA decomposition. Visual review found their existing decompositions acceptable, and we retained them.

ICA component selection

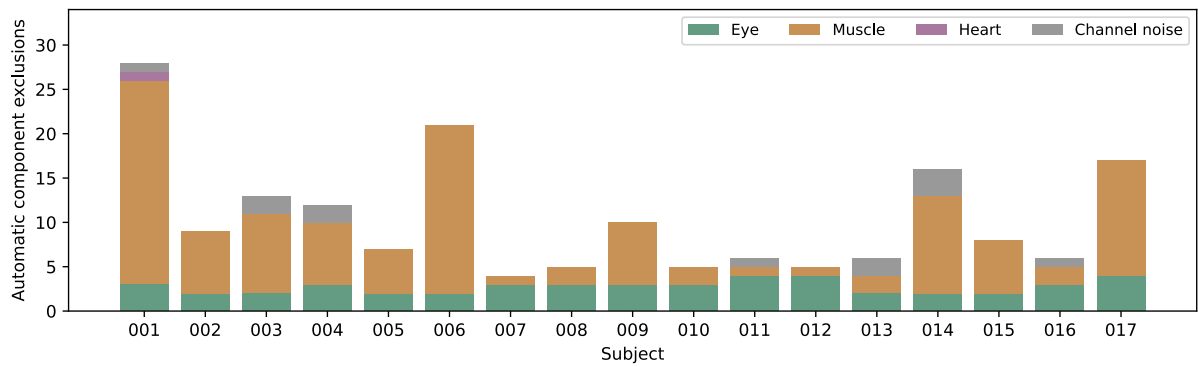


Figure 6: Automatic ICLabel exclusion proposals by subject and category, before manual additions.

ICLabel proposed 178 exclusions: 118 muscle, 47 eye, 12 channel-noise and one heart component. Across all components, there is a mean of 2.76 eye components per subject. The original paper reports one blink component per subject, and a mean of 1.76 horizontal-eye components, which also adds up to exactly 2.76 eye components per subject. [1]

Manual visual review found recognizable ocular, muscle and brain components, particularly among early components, but also 15 additional artifacts that did not pass the automatic exclusion threshold. We inspected maps, spectra and time courses and manually excluded these additional sources. The following panel illustrates one such decision.

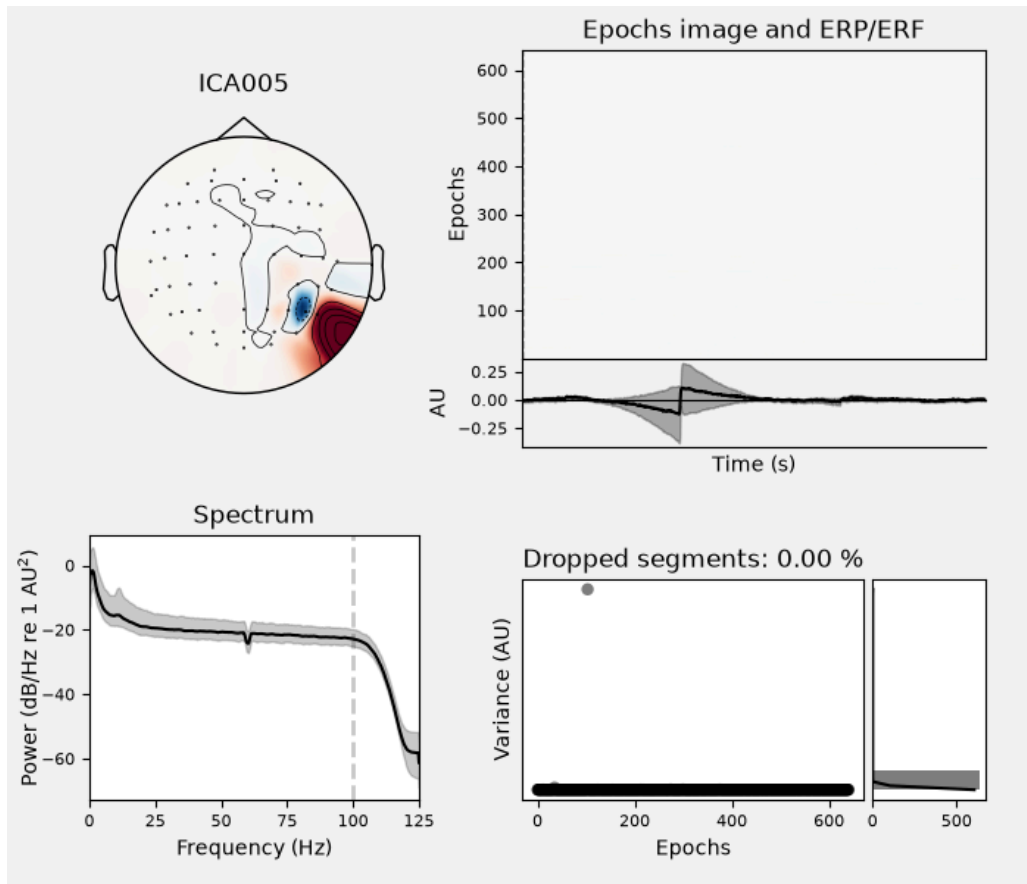


Figure 7: Manual exclusion example: subject 008, ICA005 with a disproportionately large early single-window contribution. Its channel-noise probability of 0.622 was below the automatic 0.8 threshold

With the manual additions, 193 components were approved for removal, with a median of eight per subject.

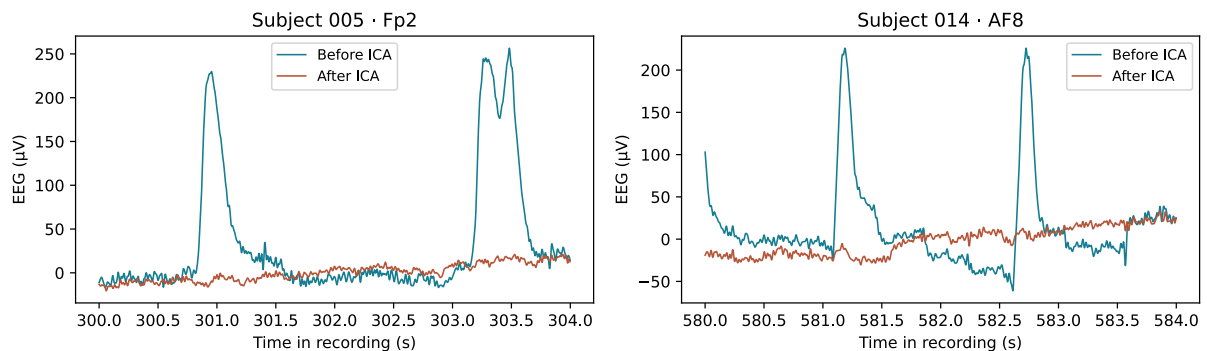


Figure 8: Effect of applying the reviewed ICA models on subjects 005 and 014. Peak-to-peak amplitude in blink-like windows decreases from approximately 273 to 42 μV and 287 to 63 μV respectively. Both curves use 0.1–40 Hz EEG and the same average reference

The first pipeline pass ends here with saved ICA models, component decisions and reports. A second pass applies those models to EEG prepared for event-related analysis.

Note about EOG channels

EOG is excluded from PyPREP, EEG ICA fitting and the average reference. The original study used EOG correlations to identify ocular components. [1] The pipeline could do this with

`ica_use_eog_detection = True`, but we instead only used ICLabel and visual review for these reasons:

- Auxiliary channels showed large, inconsistent offsets and excursions, with uncertain calibration.
- VEOG in subjects 003, 004, 006 had near-zero correlation with frontal EEG in the inspected intervals. Subject 014 had only very small correlation

EOG labels and samples remain unchanged throughout all steps and are used only for inspection. The examples below compare them with the frontal contrast Fp1–Pz.

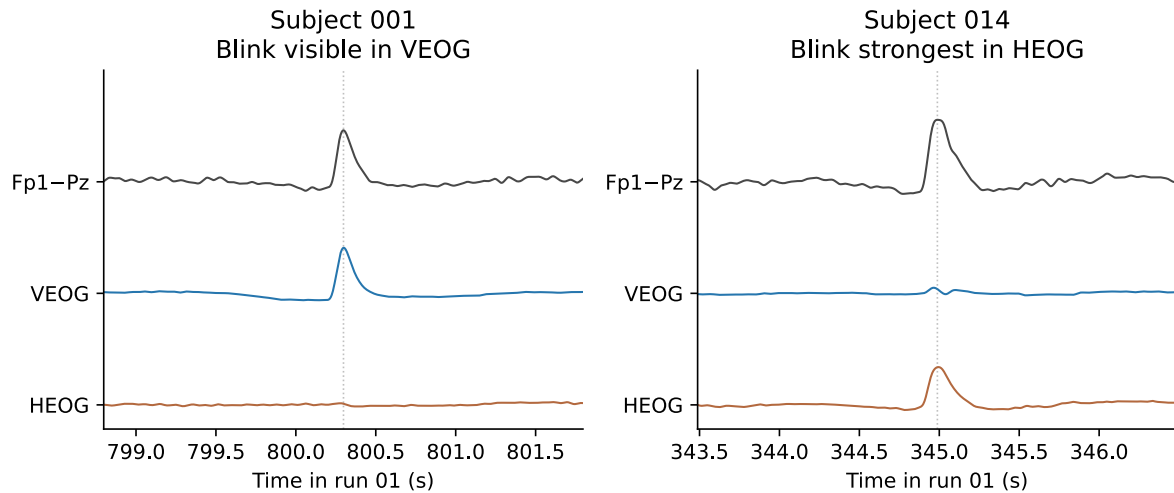


Figure 9: EOG examples from subjects 001 and 014. Visually selected three-second segments show isolated blink-like peaks. EEG was filtered 0.5–15 Hz; VEOG and HEOG share a display scale within each panel, while EEG is scaled separately. EOG polarity is aligned for display only

Second pipeline pass: from ICA models to analysis epochs

After the computation of the ICA models, we have continuous 250 Hz data that has been filtered 1–100 Hz. The analysis instead needs 500 Hz, 0.1–40 Hz EEG epoched around specific events, followed by trial cleaning and a prestimulus baseline. We therefore use a separate pipeline configuration, `config_postica.py`, reusing the reviewed models from the previous pipeline execution.

1. Select eligible events from the prepared dataset

The original paper notes that some of the game events occur in quick succession, not giving each of the game events a meaningful epoch. A look at the data confirms this issue:

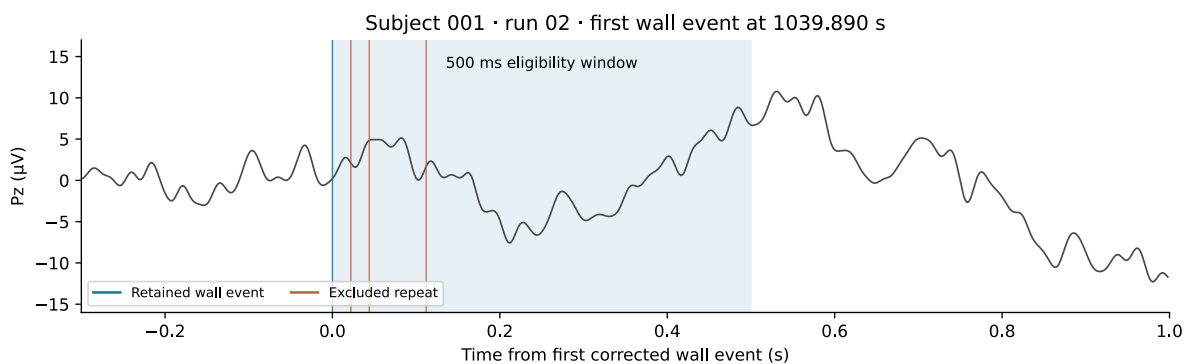


Figure 10: Repeated wall crashes in subject 001, run 02. Four events occur within 112 ms; the 500 ms rule retains the first and excludes the other three. The plot shows the original Pz signal with 0.1–40 Hz display filtering and pre-event mean removed. Time is relative to the first corrected wall event

The issue affects wall crashes (91) and star collection (93). To circumvent this, each of these events is compared to the last retained event of the same type: if the time between these events is below 500 ms, the repeat event gets marked as excluded. This is done by adding a `EXCLUDED_REPEAT_...` prefix to the affected wall/star events, allowing e.g. ERPs to ignore these. The same effective step was done in the original paper as well and affects 3,010 / 4,443 wall events and 6,556 of 13,776 star events.

2. Filter, reference and epoch the EEG

The original 500 Hz runs have to be filtered continuously to 0.1–40 Hz. Then, with `task_is_rest = False`, the new pipeline creates $-0.2...+0.8$ s epochs around the ten eligible event labels to supply the intervals used by our classifiers.

3. Apply the reviewed ICA exclusions

The pipeline loads the saved models and native component-status tables and applies the 193 approved exclusions to compatible continuous EEG and event epochs.

4. Repair or reject analysis epochs

AutoReject is now run again. Previously, it was used before ICA to select training windows. This time, using `reject = "autoreject_local"`, it can repair individual channels in retained epochs as well as reject complete trials. The following counts describe this second stage.

Of 36,258 events that are eligible for the classifiers, 14 windows extend beyond a recording boundary. AutoReject rejects 4,049/36,244 (11.2%) epochs. Among the retained epochs, 28,316 (88.0%) receive at least one channel repair (median 4). Subject medians range from one to nine. Selected interpolation limits are 4, 8 or 16 (pipeline default).

Condition	Eligible Events	Final trials
Oddball Standard	2,719	2,247
Oddball Rare	681	539
Gambling Win	618	515
Gambling Loss	617	494
Player Crash Wall	1,433	1,235
Collect Star	7,220	6,457
Missile Hit Enemy	6,495	5,923
Player Crash Enemy	1,810	1,556
Collect Ammo	1,103	1,007
Shoot Button	13,562	12,222

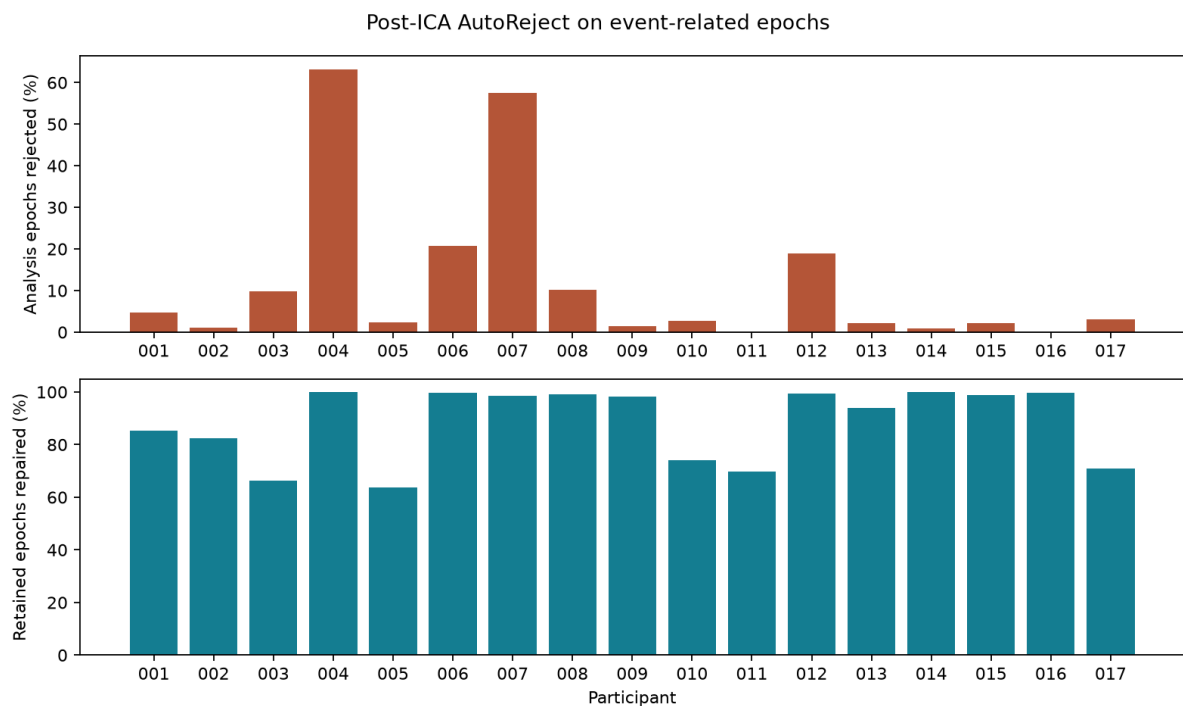


Figure 11: Post-ICA trial rejection and the percentage of retained trials with at least one channel repair.

The rejection is uneven across subjects. Subject 007 retains only 5/44 rare oddball trials and 34/156 standards. 004 retains 11/40 gambling losses and 17/40 wins. Subject 001 retains only 87/200 oddball trials despite 4.6% rejection pooled across all conditions (this is due to gameplay being a much larger contributor of epochs than oddball or gambling).

We again experimented with lowering rejection to retain more trials, but did not see an improvement in data quality.

5. Baseline correction and saved analysis outputs

After repair/rejection, the pipeline applies the $-0.2...0$ s baseline and finishes the final *_proc-clean_epo.fif files and reports. Global bad channels remain marked in these epochs.

ERP checks before and after cleaning

The following figure compares the ERPs for exemplary trials before and after our main cleaning. For the Oddball task, it can be seen that the Standard event stays much closer to the baseline after cleaning, and both events stay much more stable after 0.6 s. For the Gambling task, cleaning mostly corrects the dip seen at ~ 0.5 s from -4 uV to 0 uV.

Grand-average condition ERPs · 0.1–40 Hz

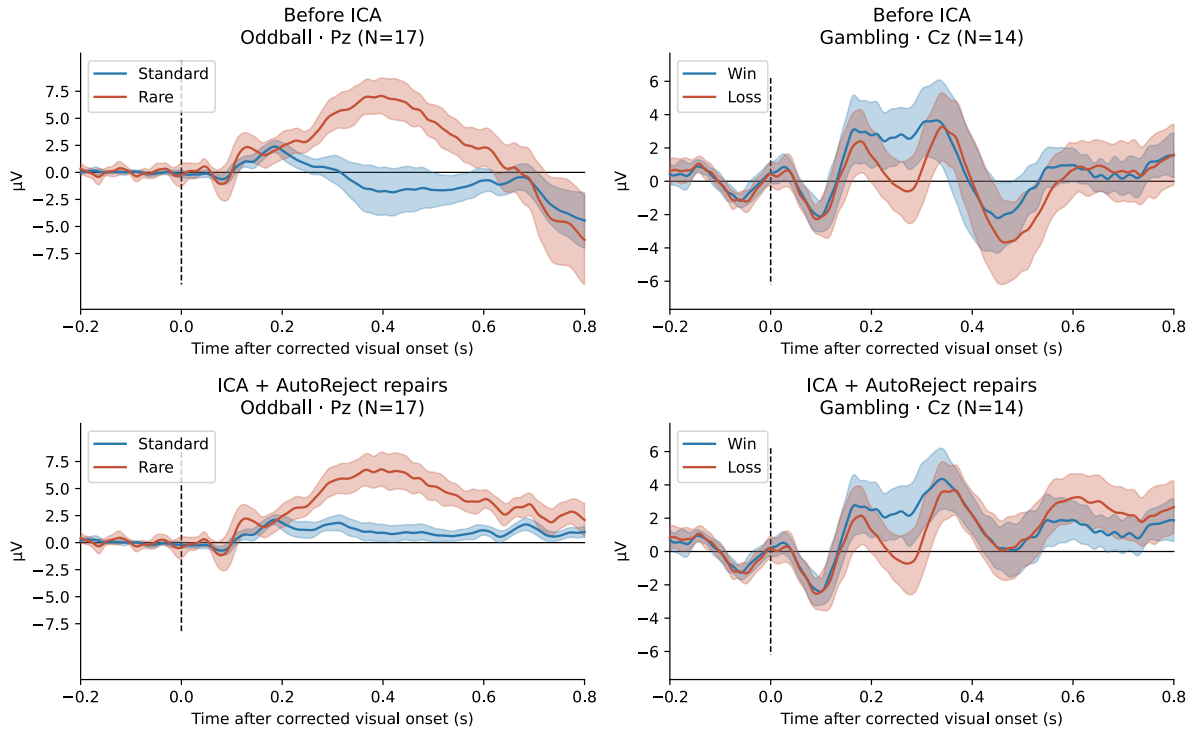


Figure 12: Grand-average condition ERPs before ICA (top) and after ICA plus final AutoReject repairs (bottom), at Pz for oddball (left, N = 17) and Cz for gambling (right, N = 14). Both rows use identical final retained trial IDs, 0.1–40 Hz filtering and –0.2...0 s baseline. Shading is the pointwise 95% t confidence interval

Analysis

Replication objective

The original study examined whether EEG patterns associated with exemplary events could be identified during continuous and complex gameplay. To do this, the authors trained participant-specific classifiers on oddball and gambling trials, then transferred them to game events. Missile hits, enemy crashes and wall crashes showed a bias towards rare-oddball classifications at later post-event times (P3). Missile hits also resembled gambling wins. However, the expected loss-like response to crashes could not be established, and crashes were partly also classified as win-like. The authors interpreted these findings as evidence that signatures from the exemplary tasks associated with salience and reward generalize to gameplay. [1]

Our objective is to see whether these relationships remain observable after our different preprocessing pipeline and a different analysis approach (using EEGNet). We therefore examine whether gameplay events are assigned the same exemplary categories as in the paper, and whether these classifications differ from those of ordinary gameplay.

Analysis design

For each test participant, we train a classifier using exemplary trials from all other participants. We then apply this classifier to the test participant's exemplary trials and gameplay events. In the original paper, classifiers were trained and transferred within the same participant instead. This means in addition to testing the generalizability across tasks, we are now testing how well the training from other subjects generalizes to other subjects as well.

The training inputs consist of the complete 0–800 ms response and two shorter intervals: 300–600 ms for oddball and 200–350 ms for gambling. The shorter intervals focus on the P3 and feedback-related responses, which were the deciding intervals for the original paper.

Classifier choice

We use Braindecode 1.5.1 and its implementation of the EEGNet model with default parameters. EEGNET is a compact convolutional network designed to learn spatial and temporal EEG features from voltage samples. We chose it due to its ease of use and small size (which suits our relatively small amount of training data). We are only supplying the input dimensions, sampling frequency and number of output classes. Training settings are specified separately in the methods appendix. [7], [8]

Input data and participant inclusion

The cleaned dataset contains 2,786 oddball trials from 17 participants: 2,247 standard and 539 rare. Fourteen participants contribute 1,009 gambling trials: 494 losses and 515 wins. These labelled exemplary trials train the classifiers that are supposed to differentiate between oddball standards and oddball rares, or between gambling wins and gambling losses.

“Cleaned dataset” refers to the data that has been resampled to 250 Hz, filtered 0.1–40 Hz with –200 to 0 ms baseline. Bad channels have been interpolated to obtain common 63 EEG inputs, EOG is excluded.

Whole-epoch classification and transfer: 0–800 ms

We use a training/test split of 16/1 participants for oddball and 13/1 participants for gambling. All trials from a participant stay in that person's set. Each model is trained for 20 epochs with a batch size of 64. The test participant remains excluded throughout. This procedure is repeated until each participant has served as the test participant once.

We run each training/test split three times with fixed random seeds, as we have observed that random seeds can again significantly impact the results we get here. Performance and transfer fractions are calculated separately for each run and then averaged within each test participant. This reduces dependence on a single random training run.

EEG values are standardized using means and standard deviations calculated from the training data only. Class weights are inversely proportional to the number of training trials in each class.

For the test participant's exemplary trials, we compare the predicted labels with the known event labels. We calculate the percentage of correctly classified trials separately for each class, then average the two percentages. Standard and rare trials, or wins and losses, contribute equally even when their trial counts differ, meaning random guessing has an expected balanced accuracy of 50%.

For gameplay events, we simply report the percentage of each event type assigned to rare by the oddball classifier or win by the gambling classifier. A model score of at least 0.5 gives the rare or win label; a lower score gives standard or loss, respectively. We compare these percentages with the same classifier's predictions on epochs sampled at random times during gameplay. These control epochs represent ordinary gameplay and may contain recorded events.

We calculate the percentages separately for each participant and then average them, giving every participant equal weight. Because subject 007 only has ten retained wall-crash trials, we decided to exclude it here (their other gameplay comparisons are retained).

Exemplary-task classification

Task	Participants	Balanced accuracy
Oddball	17	80.7%
Gambling	14	65.1%

EEGNet discriminates the conditions in the exemplary tasks for subjects whose data were not used to train the classifier, with higher mean accuracy for oddball than gambling.

Gameplay transfer

The table reports the classifier results on gameplay events. For example, for "Rare" and "Missile hit", we have 27.1% of missile-hit trials classified as rare, whereas 17% of the random control trials were classified as rare, which gives a difference of 9.8%.

Label	Game event	N	Event labels	Gameplay controls	Difference (pp)
Rare	Missile hit	17	27.1%	17.3%	+9.8
Rare	Enemy crash	17	42.5%	17.3%	+25.2
Rare	Wall crash	16	31.3%	17.0%	+14.3
Win	Missile hit	14	60.0%	48.6%	+11.3
Win	Enemy crash	14	67.8%	48.6%	+19.1
Win	Wall crash	13	58.1%	48.9%	+9.2

All three game events receive rare labels more frequently than random gameplay, although their mean rare-label fractions remain below 50%. This means the classifier seems to have a shift towards the rare category.

Missile hits also receive more win labels than random gameplay. The same direction is observed for enemy and wall crashes, which means we cannot interpret the gambling classifier as a predictor of reward. The original paper similarly could not make this inference.

The mean event-minus-control differences are positive for all six comparisons. These are descriptive results from the fitted models.

The original paper specifically identified specific shorter intervals of the epochs to be responsible for the decisions, rather than the complete 800 ms response. In the following we will again train classifiers, using only the time intervals isolated in the original study.

Comparison in the P3 and feedback intervals

We now train EEGNet using only 300–600 ms for oddball and 200–350 ms for gambling. These intervals focus on the P3 and feedback-related responses. The classifiers use the same default model settings, participant assignments and training procedure as the whole-epoch analysis.

Gameplay transfer

Within these intervals, EEGNet reaches 74.9% balanced accuracy for oddball and 60.3% for gambling, which is 5% worse for both classifiers.

The table reports transfer results for these new classifiers

Label	Game event	N	Event labels	Gameplay controls	Difference (pp)
Rare	Missile hit	17	30.8%	18.1%	+12.7
Rare	Enemy crash	17	41.3%	18.1%	+23.2
Rare	Wall crash	16	31.9%	17.6%	+14.3
Win	Missile hit	14	61.5%	45.0%	+16.5
Win	Enemy crash	14	69.8%	45.0%	+24.8
Win	Wall crash	13	59.9%	45.2%	+14.7

For oddball, all three events shift towards rare relative to random gameplay, but their mean rare-label fractions remain below 50%. We can therefore again not interpret an oddball rare as a “target-like” result. For gambling, all of the events now receive predominantly win labels, which matches the papers ambiguous result here as well.

Discussion

This project tested whether the original findings remain observable with a different preprocessing and analysis pipeline. Preparing the shared dataset required corrections to channel metadata, electrode coordinates and event timing. PyPREP, reviewed ICA removal and AutoReject reduced visible artifacts while retaining the main ERP distinctions. However, trial loss was uneven, particularly for subjects 004 and 007, and 88% of retained analysis epochs required channel repair. Successful pipeline execution therefore does not establish that cleaning was equally effective across subjects or tasks. Despite the large differences in preprocessing, many of the results, such as average number of eye components, number of electrodes that were marked as bad, and the resulting ERP were very similar.

The analysis shows partial agreement with the original findings. EEGNet distinguishes the exemplary categories for subjects whose data were not used to train the classifier. Gameplay events receive more rare labels than random gameplay, but their mean rare-label fractions remain below 50% in the P3 interval. Missile hits receive predominantly win labels in the feedback interval, matching the direction reported in the paper; crashes also shift towards win, limiting a selective reward interpretation. [1]

These results come from the same small dataset, with 17 subjects for oddball and 14 for gambling. Preprocessing uses each subject's full recordings, so classification across subjects evaluates transfer after retrospective cleaning. Preprocessing, classifier type and training population changed together; we cannot attribute discrepancies to any single choice. Moreover, transferred labels may reflect sensory or motor activity, overlapping events or residual artifacts rather than a specific psychological process.

A focused follow-up should vary one choice at a time: compare within-subject and across-subject training on the same cleaned data, then compare epoch-rejection policies with the classifier fixed. Such comparisons would help identify which findings are sensitive to the processing choices.

Conclusion

The alternative workflow recovered exemplary-task distinctions and part of the gameplay transfer pattern, but did not establish robustness of the full original findings. The project provides a documented, inspectable reanalysis and shows why preprocessing quality and analysis design must both inform the interpretation of a partial replication.

References

- [1] J. F. Cavanagh and J. Castellanos, "Identification of canonical neural events during continuous gameplay of an 8-bit style video game," *NeuroImage*, vol. 133, pp. 1–13, 2016, doi: 10.1016/j.neuroimage.2016.02.075.
- [2] J. F. Cavanagh and J. Castellanos, "EEG: Continuous gameplay of an 8-bit style video game." [Online]. Available: <https://openneuro.org/datasets/ds003517>
- [3] Okimoka, "EEG_VL: accompanying project repository." [Online]. Available: https://github.com/Okimoka/EEG_VL
- [4] BIDS contributors, "Brain Imaging Data Structure: Electroencephalography." [Online]. Available: <https://bids-specification.readthedocs.io/en/stable/modality-specific-files/electroencephalography.html>
- [5] PyPREP contributors, "PyPREP: NoisyChannels reference." [Online]. Available: <https://pyprep.readthedocs.io/en/latest/generated/pyprep.NoisyChannels.html>

- [6] MNE-ICALabel contributors, “Repairing artifacts with ICA automatically using ICLabel.” [Online]. Available: https://mne.tools/mne-icalabel/stable/generated/examples/00_iclabel.html
- [7] V. J. Lawhern, A. J. Solon, N. R. Waytowich, S. M. Gordon, C. P. Hung, and B. J. Lance, “EEGNet: A Compact Convolutional Neural Network for EEG-based Brain–Computer Interfaces.” [Online]. Available: <https://arxiv.org/abs/1611.08024>
- [8] Braindecode contributors, “Braindecode: EEGNet implementation.” [Online]. Available: <https://braindecode.org/stable/generated/braindecode.models.EEGNet.html>